

Performance Testing

Date	30 September 2026
Team ID	6aba2e98ca93a2750963fe65
Project Name	PocketSmart AI - Your Smart Budget & Recommendation Assistant
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Locust (load-testing tool)
Type of Testing	Load Testing - planned / documented
Target Module / API	/generate-home, /generate-party, /generate-jewelry, /history
Test Environment	Local FastAPI / Uvicorn environment
Test Date	30 September 2026 - performance test plan documented; execution evidence pending

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Home Planner API - /generate-home	10	60	Stable responses; avg response time target < 2s; error rate < 1%
2	Party Planner API - /generate-party	10	60	Stable responses; avg response time target < 2s; error rate < 1%
3	Jewelry Planner API - /generate-jewelry with optional image	5	60	Image request handled; max response target < 5s; error rate < 1%
4	Authentication + Recommendation History	20	60	Login/history endpoints remain responsive without critical errors

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	Not measured	N/A	Execution evidence not provided
2	Response Time (Max)	< 5 seconds	Not measured	N/A	Execution evidence not provided
3	Throughput (Req/sec)	>= 5 req/sec	Not measured	N/A	Planned target; validate with load run
4	Error Rate	< 1%	Not measured	N/A	Execution evidence not provided
5	CPU Utilization	< 80%	Not measured	N/A	Execution evidence not provided
6	Memory Utilization	< 80%	Not measured	N/A	Execution evidence not provided

Step 4: Observations & Analysis

Key Findings:

The performance test plan covers the main PocketSmart AI FastAPI flows: Home, Party, Jewelry, and history/authentication endpoints. Actual runtime measurements were not supplied in the source material.

Bottlenecks Identified:

Potential load points are Gemini AI recommendation generation and optional Jewelry image processing; these should be monitored during an actual Locust run.

Optimization Steps Taken:

Use modular FastAPI endpoints, request validation, recommendation history storage, and fallback recommendation logic. Run load testing before deployment and optimize slow AI/API paths based on measured results.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

Performance evidence not provided in the source files.

Attach Locust/JMeter/k6 results after executing the planned load tests.

Suggested evidence: average/max response time, throughput, error rate, CPU and memory usage.